



INSTALLATION AND OPERATION GUIDE

A DEEP LEARNING APPROACH FOR ARTWORK RECONSTRUCTION

(An extended framework based on Mask-Aware Transformer - MAT)

Student: Ngo Thi Thuong
Student ID: ITCSIU21160
Thesis Advisor: Le Thi Ngoc Hanh, Ph.D
Submission Date: June 2026

Tài liệu này cung cấp hướng dẫn chi tiết cách thiết lập môi trường, chạy thử nghiệm hệ thống khôi phục tranh chân dung nghệ thuật bị hỏng (FaceArt Restoration) dựa trên mô hình học sâu đề xuất dựa trên kiến trúc gốc Mask-Aware Transformer (MAT). Tài liệu mô tả đầy đủ từ cấu trúc dự án, cấu hình môi trường cho GPU NVIDIA, MacBook M1/M2/M3 (MPS), chạy giao diện đồ họa tương tác Streamlit UI, dòng lệnh CLI, cho đến quy trình huấn luyện lại mô hình qua 3 giai đoạn độc lập để tái tạo kết quả.

I. Tổng quan Hệ thống & Cấu trúc Mã nguồn

1. Giới thiệu Hệ thống

Hệ thống này khôi phục các bức tranh vẽ chân dung nghệ thuật bị hư hại (trầy xước, rách nứt, mất chi tiết cấu trúc) thông qua mô hình học sâu đề xuất dựa trên kiến trúc gốc Mask-Aware Transformer (MAT). Khác với các phương pháp khôi phục ảnh thông thường, mô hình này được tinh chỉnh chuyên biệt cho đặc trưng gương mặt chân dung nghệ thuật bằng cách sử dụng các cơ chế Relative Position Bias, Transformer Adapters, và Structure Guidance để giữ nguyên các nét cọ nghệ thuật và độ chính xác của ngũ quan gương mặt.

2. Cấu trúc Thư mục Dự án

Dưới đây là sơ đồ cấu trúc thư mục cốt lõi của project và chức năng của từng thư mục/tệp tin chính để người đọc dễ dàng định vị:

- [checkpoints/](file:///d:/2025-2026/Thesis/Clone/MAT/checkpoints) — Thư mục lưu trữ các file trọng số mô hình (.pkl) sau khi huấn luyện xong.
- [collab/](file:///d:/2025-2026/Thesis/Clone/MAT/collab) — Các kịch bản shell và python hỗ trợ chạy huấn luyện/suy luận trên Colab hoặc Server.
- [run_phase1_server_from_drive.sh](file:///d:/2025-2026/Thesis/Clone/MAT/collab/run_phase1_server_from_drive.sh) — Kịch bản tự động tải dữ liệu & huấn luyện Phase 1.
- [run_phase3_server_from_phase2.sh](file:///d:/2025-2026/Thesis/Clone/MAT/collab/run_phase3_server_from_phase2.sh) — Kịch bản tự động kế thừa Phase 2 & huấn luyện Phase 3.
- [datasets/](file:///d:/2025-2026/Thesis/Clone/MAT/datasets) — Mã nguồn quản lý luồng dữ liệu (Data loaders) và sinh mặt nạ hư hại.
- [dataset_512.py](file:///d:/2025-2026/Thesis/Clone/MAT/datasets/dataset_512.py) — Bộ nạp dữ liệu huấn luyện cho ảnh phân giải 512x512.
- [mask_generator_512.py](file:///d:/2025-2026/Thesis/Clone/MAT/datasets/mask_generator_512.py) — Công cụ sinh mặt nạ ngẫu nhiên mô phỏng vùng hư hại dạng khối.
- [demo_ui/](file:///d:/2025-2026/Thesis/Clone/MAT/demo_ui) — Mã nguồn hỗ trợ xây dựng giao diện Streamlit.
- [inference_adapter.py](file:///d:/2025-2026/Thesis/Clone/MAT/demo_ui/inference_adapter.py) — Cầu nối trung gian gọi suy luận cục bộ (local) hoặc từ xa (remote FastAPI).
- [parf_compare.py](file:///d:/2025-2026/Thesis/Clone/MAT/demo_ui/parf_compare.py) — Lập trình logic cho tính năng so sánh kết quả khôi phục.
- `dnnlib` — Thư viện tiện ích cấp thấp của StyleGAN2-ADA dùng để quản lý cấu hình và log.

- [evaluation/](file:///d:/2025-2026/Thesis/Clone/MAT/evaluation) — Các script python dùng để đánh giá định lượng chất lượng ảnh khôi phục.
- `eval\_psnr\_ssim.py` — Đo chỉ số độ tương đồng cấu trúc SSIM và chỉ số PSNR.
- `eval\_fid.py` — Đo khoảng cách phân phối ảnh sinh FID.
- [losses/](file:///d:/2025-2026/Thesis/Clone/MAT/losses) — Định nghĩa các hàm mất mát (Loss functions) sử dụng khi huấn luyện.
- [loss.py](file:///d:/2025-2026/Thesis/Clone/MAT/losses/loss.py) — Hàm mất mát `TwoStageLoss` chính của mô hình MAT.
- [focal_frequency_loss.py](file:///d:/2025-2026/Thesis/Clone/MAT/losses/focal_frequency_loss.py) — Hàm mất mát tần số tiêu điểm giúp khôi phục chi tiết họa tiết nét vẽ.
- [networks/](file:///d:/2025-2026/Thesis/Clone/MAT/networks) — Định nghĩa cấu trúc mạng nơ-ron học sâu của hệ thống.
- [mat.py](file:///d:/2025-2026/Thesis/Clone/MAT/networks/mat.py) — Cấu trúc mạng chính Mask-Aware Transformer cùng các khối `WindowAttention` sửa đổi.
- [structure_guidance.py](file:///d:/2025-2026/Thesis/Clone/MAT/networks/structure_guidance.py) — Khối dẫn đường cấu trúc hình học chân dung tích hợp ở Phase 3.
- `torch\_utils/` — Các module tiện ích tích hợp sâu với PyTorch hỗ trợ tăng tốc tính toán.
- `training/` — Chứa vòng lặp huấn luyện chính (training loop) và bộ tinh chỉnh tham số học.
- [dataset_tool.py](file:///d:/2025-2026/Thesis/Clone/MAT/dataset_tool.py) — Công cụ tiền xử lý ảnh thô thành file zip huấn luyện không nén.
- [generate_image.py](file:///d:/2025-2026/Thesis/Clone/MAT/generate_image.py) — Script suy luận nhanh thông qua giao diện dòng lệnh (CLI).
- [requirements.txt](file:///d:/2025-2026/Thesis/Clone/MAT/requirements.txt) — Danh sách toàn bộ các thư viện Python phụ thuộc cần cài đặt.
- [streamlit_app.py](file:///d:/2025-2026/Thesis/Clone/MAT/streamlit_app.py) — Entrypoint chính khởi chạy giao diện Web tương tác Streamlit.

II. Yêu cầu Hệ thống & Cài đặt Môi trường

1. Yêu cầu Phần cứng (Hardware Requirements)

- ****Dành cho Suy luận (Inference) và Chạy Giao diện UI:****
- ****Hệ điều hành:**** Windows 10/11, macOS Big Sur trở lên (hỗ trợ cả Intel & Apple Silicon M1/M2/M3), hoặc Linux Ubuntu 18.04+.
- ****RAM:**** Tối thiểu 16 GB.
- ****Thiết bị tăng tốc đồ họa:****
- ****Windows/Linux:**** GPU NVIDIA hỗ trợ CUDA (VRAM tối thiểu 6-8 GB).
- ****MacBook:**** Apple Silicon M-series (sử dụng MPS - Metal Performance Shaders thông qua GPU tích hợp).
- ****CPU-only:**** Hỗ trợ trên tất cả các dòng máy (tốc độ chậm hơn).
- ****Dành cho Huấn luyện lại mô hình (Training):****
- ****Hệ điều hành:**** Linux Ubuntu 20.04/22.04 LTS (Khuyến nghị).
- ****GPU:**** GPU NVIDIA chuyên dụng (RTX 3090, RTX 4090, A100, RTX A6000) có VRAM từ 16GB đến 24GB trở lên.
- ****CPU:**** Intel Xeon hoặc AMD EPYC (số lõi ≥ 8).

2. Các bước thiết lập Môi trường ảo (Environment Setup)

Khuyến nghị sử dụng Conda hoặc Miniconda để quản lý môi trường ảo giúp tránh các xung đột thư viện:

Bước 1: Tạo môi trường ảo Python 3.8

```
conda create -n faceart_env python=3.8 -y
conda activate faceart_env
```

- ****Cách khác (Nếu dùng Python venv truyền thống):****
- ***Trên Windows (PowerShell):***

```
python -m venv .venv
.venv\Scripts\Activate.ps1
```

- ***Trên Linux/macOS:***

```
python3 -m venv .venv
source .venv/bin/activate
```

Bước 2: Cài đặt thư viện PyTorch

Chọn lệnh cài đặt phù hợp nhất với cấu hình thiết bị của bạn:

- ****Đối với hệ máy chạy GPU NVIDIA (Hỗ trợ CUDA 11.8 - Khuyến nghị):****

```
pip install torch==2.1.2 torchvision==0.16.2 --index-url
https://download.pytorch.org/whl/cu118
```

- ****Đối với Apple Silicon MacBook (MPS - M1/M2/M3):****

```
pip install torch torchvision
```

- ****Đối với hệ máy chỉ chạy CPU (CPU-only):****

```
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
```

Bước 3: Cài đặt các thư viện hỗ trợ khác và Ninja

Cài đặt toàn bộ các thư viện được liệt kê trong `requirements.txt`:

```
pip install --upgrade pip
pip install -r requirements.txt
```

*****Về trình biên dịch Ninja:*****

Thư viện `ninja` là bắt buộc để build các toán tử tối ưu CUDA cho mô hình MAT. Đối với Windows/Linux chạy GPU, hãy cài đặt Ninja bằng lệnh:

** Dùng Conda: `conda install -c conda-forge ninja -y`*

** Dùng Pip: `pip install ninja`*

Đối với CPU hoặc MacBook (MPS), nếu gặp lỗi biên dịch custom ops, mã nguồn sẽ tự động chuyển sang chế độ CPU fallback tương thích.

III. Hướng dẫn Vận hành Giao diện Streamlit Web UI

Mã nguồn luận văn cung cấp một giao diện Web tương tác hoàn chỉnh thông qua Streamlit (`streamlit\_app.py`) giúp người dùng tải ảnh chân dung bị hỏng lên, vẽ mask, và tiến hành phục hồi qua từng giai đoạn trực quan.

1. Khởi chạy ứng dụng Streamlit UI

Từ thư mục gốc của project, chạy lệnh sau trong terminal đã kích hoạt môi trường ảo:

```
streamlit run streamlit_app.py --server.port 8501 --server.address 127.0.0.1
```

Sau khi khởi chạy thành công, ứng dụng sẽ mở ra trên trình duyệt tại địa chỉ: [\[http://127.0.0.1:8501\]](http://127.0.0.1:8501)(<http://127.0.0.1:8501>)

2. Cấu hình Backend & Thiết bị (Advanced Controls)

Nhấn mở mục **Advanced Controls** ở cột bên trái để thiết lập các thông số suy luận:

- **Chế độ A: Local Backend (Chạy trực tiếp trên máy của bạn)**
- Phù hợp khi máy cá nhân có GPU NVIDIA hoặc khi chạy trực tiếp trên MacBook.
- Tải các file checkpoints (`.pkl`) cần thiết đặt vào thư mục `checkpoints/`:
 - Checkpoint Stage 1: `resume\_phase1\_from\_finetune\_plus\_loss.pkl`
 - Checkpoint Giai đoạn cuối (Final / Phase 2-3): `network-snapshot-000072.pkl`
 - Checkpoint MAT Gốc (Baseline): `Places\_512\_FullData.pkl`
- Trên UI:
 - Đặt `Backend Mode` = `local`.
 - Đặt `Device` = `cuda` (nếu chạy GPU NVIDIA) hoặc `cpu`.
 - Đối với **MacBook (Apple Silicon M1/M2/M3)**, nhập thủ công chữ `"mps"` vào ô cấu hình thiết bị để tăng tốc bằng GPU tích hợp.
 - Nhập đường dẫn tương ứng của các file checkpoint cục bộ đã tải vào các ô nhập liệu tương ứng.
- **Chế độ B: Remote Backend (Chạy qua API Server của máy GPU ngoài - Ví dụ Colab/Server)**
- Tiện lợi khi máy tính của bạn yếu (ví dụ MacBook cấu hình thấp) nhưng muốn trải nghiệm giao diện mượt mà bằng cách đẩy tác vụ tính toán nặng cho một máy chủ GPU khác qua mạng.
- Trên máy chủ GPU (hoặc Google Colab), khởi động API Server bằng FastAPI:

```
uvicorn colab_inference_api:app --host 0.0.0.0 --port 8000
```

- Trên Streamlit UI ở máy khách (MacBook/PC):
 - Đặt `Backend Mode` = `remote`.
 - Nhập địa chỉ API Server vào ô `Remote endpoint` (Ví dụ: `http://127.0.0.1:8000` hoặc đường dẫn ngrok công khai được sinh ra từ Colab).

3. Quy trình Trải nghiệm trên Giao diện (Streamlit UI Flow)

Giao diện Streamlit được phân chia thành 4 bước dọc theo quy trình làm việc của mạng MAT:

1. **Input&Mask (Tạo dữ liệu đầu vào):** Tải lên bức ảnh chân dung bị hỏng. Người dùng có thể tạo mặt nạ vùng hỏng bằng cách chọn các preset vị trí (Center, Left, Right, v.v.), điều chỉnh kích thước hoặc vẽ tự do trên tranh bằng công cụ vẽ tay (Drawable Canvas). Dùng các nút căn chỉnh (← Left, → Right, ↑ Up, ↓

Down) để định vị vết nứt chính xác trên khuôn mặt.

2. ****Masked Input (Xác nhận đầu vào):**** Hiển thị ảnh chân dung gốc kèm theo lớp phủ xám đen biểu thị khu vực hư hại sẽ được khôi phục. Nhấn chọn "Confirm Input Package" để khóa cấu hình.
3. ****Stage 1 (Phục hồi thô):**** Sử dụng mô hình được huấn luyện ở Phase 1 (FaceArt Finetune + Reconstruction Loss) để tạo ra một ảnh khôi phục sơ bộ lấp đầy các khoảng trống cấu trúc lớn.
4. ****Final Output (Kết quả khôi phục hoàn thiện):**** Sử dụng mô hình được huấn luyện ở Phase 2 & 3 (Tích hợp Transformer Adapters, Structure Guidance và Adaptive Gate) để khôi phục hoàn chỉnh các chi tiết kết cấu siêu nhỏ (các đường nét mắt, mũi, miệng, nét cọ vẽ) sắc nét nhất.
5. ****So sánh trực quan (Compare View & Lightbox):**** Chọn tab ****Compare**** để xếp song song các ảnh kết quả: Ảnh gốc hư hại, Ảnh khôi phục thô (Stage 1), Ảnh khôi phục hoàn thiện (Final Output) và Ảnh khôi phục từ mô hình MAT Baseline (gốc chưa tinh chỉnh). Dùng chế độ ****Lightbox**** để zoom (scroll chuột) và pan (kéo thả chuột) nhằm so sánh chi tiết cọ vẽ cực nhỏ.

IV. Hướng dẫn Sử dụng Suy luận bằng Dòng lệnh (CLI Inference)

Để suy luận hàng loạt cho nhiều ảnh và mask trong thư mục mà không cần mở giao diện đồ họa, sử dụng tệp lệnh `generate\_image.py`:

```
python generate_image.py \
  --network checkpoints/network-snapshot-000072.pkl \
  --dpath test_sets/CelebA-HQ/images \
  --mpath test_sets/CelebA-HQ/masks \
  --outdir samples_output
```

****Giải thích các tham số chính:****

- `--network`: Đường dẫn đến file checkpoint mô hình đã huấn luyện (`.pkl`).
- `--dpath`: Thư mục chứa các ảnh chân dung đầu vào bị hỏng cần khôi phục.
- `--mpath`: (Tùy chọn) Thư mục chứa các ảnh mặt nạ đen trắng biểu thị vùng hỏng (0/vùng đen: giữ nguyên; 1/vùng trắng: vùng hỏng cần khôi phục). Nếu bỏ qua, hệ thống tự sinh mặt nạ ngẫu nhiên.
- `--outdir`: Thư mục đầu ra lưu ảnh chân dung đã được khôi phục.

****Chú ý về kích thước ảnh:****

Mô hình MAT chỉ hỗ trợ kích thước ảnh đầu vào là bội số của 512 (ví dụ: 512x512, 1024x1024). Nếu ảnh của bạn có kích cỡ khác, hãy sử dụng công cụ để resize hoặc pad ảnh về bội số của 512 với phần mask pad bằng 0. Trên Streamlit UI, cơ chế tự động resize hai chiều đã được tích hợp sẵn.

V. Quy trình Huấn luyện lại Mô hình (Model Retraining)

Quá trình huấn luyện lại mô hình khôi phục tranh chân dung nghệ thuật được thực hiện thông qua 3 giai đoạn độc lập nối tiếp nhau:

1. Chuẩn bị dữ liệu huấn luyện (Dataset Preparation)

Dữ liệu huấn luyện thô phải được đóng gói thành tệp nén ZIP không nén nhằm tăng tốc độ đọc dữ liệu (I/O) trong quá trình huấn luyện:

```
python dataset_tool.py --source /path/to/raw/images --dest datasets/faceart_train.zip --width 512 --height 512
```

Tương tự, tạo 2 bộ dữ liệu:

- Bộ huấn luyện (Training Set): `datasets/faceart\_train.zip`
- Bộ kiểm thử (Validation Set): `datasets/faceart\_val.zip`

2. Các giai đoạn huấn luyện (Three-Phase Training Flow)

GIẢI ĐOẠN 1: Phase 1 - Base Finetuning

- **Mục tiêu:** Tinh chỉnh mạng MAT trên tập tranh chân dung nghệ thuật FaceArt bằng cách tích hợp Relative Position Bias (Window-based Attention), Mask-Aware Additive Bias, và Deterministic Latent Gate (ổn định hóa quá trình trộn đặc trưng latent).
- **Tệp kịch bản tự động hóa trên server:** `collab/run\_phase1\_server\_from\_drive.sh`
- **Khởi chạy chuẩn bị dữ liệu và checkpoint nền:**

```
bash collab/run_phase1_server_from_drive.sh prepare
```

- **Chạy thử nghiệm (Dry-run):**

```
bash collab/run_phase1_server_from_drive.sh dry-run
```

- **Khởi chạy huấn luyện chính thức (chạy ngầm trong tmux):**

```
tmux new -d -s phase1_job "bash collab/run_phase1_server_from_drive.sh train"
```

- **Câu lệnh huấn luyện thực thi của Phase 1:**

```
python train.py \
  --outdir=runs/faceart_phasel_relbias_gate \
  --data=datasets/faceart_train.zip \
  --data_val=datasets/faceart_val.zip \
  --cfg=places512 \
  --resume=checkpoints/Places_512_FullData.pkl \
  --epochs=40 \
  --batch=4 \
  --workers=2 \
  --snap=5 \
  --metrics=None \
  --pr=0.1 \
  --pl=False \
  --truncation=0.5 \
  --style_mix=0.5 \
  --ema=10 \
  --lr=5e-5 \
  --lrt=1e-4 \
  --lambda-ffl=0.02 \
  --aug=noaug \
  --enable-rel-pos-bias=True \
  --enable-mask-bias=True \
  --enable-deterministic-latent-gate=True
```

GIAI ĐOẠN 2: Phase 2 - Transformer Adapter Training

- **Mục tiêu:** Đóng băng các tham số nền tảng của MAT và chèn thêm các mô-đun nhỏ thích ứng (Transformer Adapters ở độ phân giải 32x32 và 16x16) giúp mô hình thích nghi nhanh với phong cách tranh chân dung nghệ thuật mà không phá hủy tính tổng quát hóa của mạng MAT gốc.
- **Câu lệnh huấn luyện Phase 2:**

```
python train.py \
  --outdir=runs/faceart_phase2_tran_adapter \
  --data=datasets/faceart_train.zip \
  --data_val=datasets/faceart_val.zip \
  --cfg=places512 \
  --resume=runs/faceart_phase1_relbias_gate/00000-places512/network-snapshot-000040.pkl \
  --epochs=30 \
  --batch=4 \
  --workers=2 \
  --snap=5 \
  --metrics=None \
  --pr=0.1 \
  --pl=False \
  --truncation=0.5 \
  --style_mix=0.5 \
  --ema=10 \
  --lr=2.5e-5 \
  --lrt=1e-4 \
  --lambda-ffl=0.02 \
  --aug=noaug \
  --enable-rel-pos-bias=True \
  --enable-mask-bias=True \
  --enable-deterministic-latent-gate=True \
  --enable-tran-adapter-32=True \
  --enable-tran-adapter-16=True
```

GIẢI ĐOẠN 3: Phase 3 - Structure Guidance & Adaptive Gate

- **Mục tiêu:** Kích hoạt cơ chế dẫn đường cấu trúc hình học chân dung (Structure Guidance) để tái tạo chính xác các vùng hư hỏng siêu rộng, đồng thời huấn luyện cổng thích ứng (Adaptive Gate) để kết hợp hài hòa thông tin cấu trúc chân dung tổng thể và thông tin chi tiết cục bộ từ adapters.
- **Tập kịch bản tự động hóa trên server:** `collab/run\_phase3\_server\_from\_phase2.sh`
- **Khởi chạy chuẩn bị dữ liệu và checkpoint nền từ Phase 2:**

```
bash collab/run_phase3_server_from_phase2.sh prepare
```

- **Chạy thử nghiệm (Dry-run):**

```
bash collab/run_phase3_server_from_phase2.sh dry-run
```

- **Khởi chạy huấn luyện chính thức (chạy ngầm trong tmux):**

```
tmux new -d -s phase3_job "bash collab/run_phase3_server_from_phase2.sh train"
```

- **Câu lệnh huấn luyện thực thi của Phase 3:**

```
python train.py \
  --outdir=runs/faceart_phase3_adaptive_structure_guidance \
  --data=datasets/faceart_train.zip \
  --data_val=datasets/faceart_val.zip \
  --cfg=places512 \
  --resume=runs/faceart_phase2_tran_adapter/00000-places512/network-snapshot-000030.pkl \
  --epochs=10 \
  --batch=4 \
  --workers=2 \
  --snap=2 \
  --metrics=None \
  --pr=0.1 \
  --pl=False \
  --truncation=0.5 \
  --style_mix=0.5 \
  --ema=10 \
  --lr=5e-5 \
  --lrt=1e-4 \
  --lambda-ffl=0.02 \
  --aug=noaug \
  --enable-rel-pos-bias=True \
  --enable-mask-bias=True \
  --enable-deterministic-latent-gate=True \
  --enable-tran-adapter-32=True \
  --enable-tran-adapter-16=True \
  --enable-structure-guidance=True \
  --enable-structure-fuse-16=True \
  --enable-structure-fuse-stage2=True \
  --enable-structure-fuse-32=False \
  --enable-adaptive-structure-gate=True
```

3. Cách kiểm tra tiến trình huấn luyện (Monitoring)

Để kiểm tra nhật ký ghi chép (training logs) và các chỉ số lỗi (losses) trong quá trình huấn luyện:

- Đọc file ghi log chính của `train.py` tại thư mục chạy mới nhất:

```
tail -f runs/faceart_phase3_adaptive_structure_guidance/00000-places512/log.txt
```

- Hoặc xem nhật ký shell khởi động:

```
tail -f runs/faceart_phase3_adaptive_structure_guidance/server-launch.log
```

VI. Quy trình Đánh giá Mô hình (Metrics Evaluation)

Sau khi hoàn tất huấn luyện, bạn cần đánh giá chất lượng phục hồi bằng các chỉ số định lượng quen thuộc (FID, PSNR, SSIM, LPIPS, v.v.). Các tệp đánh giá nằm trong thư mục `evaluation/` và `thesis\_metrics/`.

1. ****Đo đặc chỉ số PSNR, SSIM và L1:****

```
python evaluation/eval_psnr_ssim.py --rec_path /path/to/reconstructed/images --gt_path /path/to/ground_truth/images
```

2. ****Đo đặc chỉ số cảm nhận LPIPS:****

```
python evaluation/eval_lpips.py --rec_path /path/to/reconstructed/images --gt_path /path/to/ground_truth/images
```

3. ****Đo đặc chỉ số FID (Fréchet Inception Distance):****

```
python evaluation/eval_fid.py --rec_path /path/to/reconstructed/images --gt_path /path/to/ground_truth/images
```